

Payment.java - Deep Dive Explanation

1. OVERVIEW & LINKAGE

This file is the Abstract Base Class. It sits at the top of the architectural hierarchy. It does not process payments on its own—instead, it acts as a blueprint. It links directly to `CreditCardPayment`, `PayPalPayment`, and `BankTransferPayment` because they all use the "extend" keyword to inherit from it. It links to `PaymentProcessor` because the processor holds a List of these generic Payment objects, rather than holding lists of specific credit cards.

2. DETAILED CODE BREAKDOWN

```
private static int ticketCounter = 1000;
```

This is a static variable. The word "static" means this variable belongs to the class itself, not to any individual payment object. It acts as a global counter shared across every single payment to ensure every transaction gets a unique, incrementing ID (TXN1000, TXN1001, etc.).

```
protected String paymentId;  
protected double amount;  
protected String status;
```

The "protected" keyword is crucial for Encapsulation. It means these variables are hidden from the outside world (like the `PaymentProcessor`), but they are fully accessible to child classes (like `CreditCardPayment`) so they can update the status to SUCCESS or FAILED.

```
public Payment(double amount, String currency) { ... }
```

This is the Constructor. It runs whenever a child class calls `super()`. It validates that the amount is positive, generates a unique ID by appending and incrementing the `ticketCounter`, and captures the exact `LocalDateTime` the transaction started.

```
public abstract boolean process(BiConsumer<String, String> log);
```

This is the core of the polymorphism algorithm. The "abstract" keyword means there is no code block here. It is a strict architectural rule that says: "Any class that extends Payment MUST write their own version of this method." The `BiConsumer` is a functional interface callback that allows the payment to pass live log messages back to the user interface.